

**BABEȘ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER
SCIENCE
SPECIALIZATION COMPUTER SCIENCE**

DIPLOMA THESIS

Assessment of Physical Rehabilitation Exercises using Deep Learning and Computer Vision

**Supervisor
Lect. Dr. Iuliana Bocicor**

*Author
Alexe Nicolae Răzvan*

2026

**UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ**

LUCRARE DE LICENȚĂ

Evaluarea exercițiilor de reabilitare fizică folosind Deep Learning și Computer Vision

**Conducător științific
Lect. Dr. Iuliana Bocicor**

*Absolvent
Alexe Nicolae Răzvan*

2026

ABSTRACT

Physical rehabilitation is critical for restoring functional mobility, yet its efficacy is severely constrained by the lack of expert supervision in home environments. While automated motion assessment systems present a scalable solution, traditional models frequently operate as opaque “black boxes.” They struggle with interpretability and often fail to account for the natural anthropometric variability among patients.

This thesis introduces KinetoCheck, an AI-assisted platform designed to evaluate movement quality using standard RGB and RGB-D cameras. The system leverages a dual-stream deep learning architecture: one stream utilizes Spatio-Temporal Graph Attention Networks (ST-GAT) to analyze spatial body topology, while a 1D-CNN stream captures joint-angle temporal dynamics. Trained on the University of Idaho Physical Rehabilitation Movement Data (UI-PRMD), the model employs triplet margin loss and Batch-Hard mining to establish a robust, subject-invariant standard for movement quality. Transcending conventional binary classification, KinetoCheck evaluates patients on a continuous quality scale. Crucially, the system exploits graph attention weights to visually isolate “Joints of Attention” (JoA), providing clinicians and patients with transparent, biomechanically interpretable feedback regarding specific movement errors.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	1
1.3	Aims and Objectives	2
2	State of the Art and Theoretical Background	3
2.1	Theoretical Foundations	3
2.1.1	Motion Analysis and Human Pose Estimation	3
2.1.2	Skeleton-Based Learning for Time Series	4
2.1.3	Graph Neural Networks and ST-GAT for Rehabilitation Assessment	6
2.1.4	Siamese Networks and Pairwise Metric Learning	7
2.1.5	Explainability in Movement Assessment	7
2.2	Review of Related Work	8
2.2.1	Traditional Movement Assessment	8
2.2.2	The Pioneer Deep Learning Framework (Liao et al.)	9
2.2.3	Contemporary GCN and Transformer Models	9
2.2.4	Video-Based and Smartphone Telerehabilitation	9
2.3	Critical Synthesis and Research Gaps	9
3	Application Requirements and System Architecture	11
3.1	System Scope and Problem Definition	11
3.2	System Requirements	11
3.2.1	Functional Requirements	11
3.2.2	Non-Functional Requirements	12
3.3	Architectural Overview	13
3.3.1	Model and Inference Design	16
3.3.2	Data Flow and Artifact Specification	16
3.3.3	Analytics and Reporting Layer	17
3.4	Software Design Patterns	17
3.4.1	Factory and Singleton Cache Patterns	17

3.4.2	Composition and Component Isolation	17
4	Training Design and Software Implementation	18
4.1	Model Training Strategy	18
4.1.1	Spatio-Temporal Graph Attention Network (ST-GAT) Architecture	18
4.1.2	Mathematical Formulation of the Contrastive Objective	20
4.2	Data Processing and Normalization	22
4.2.1	Identity Leakage Mitigation	22
4.2.2	Kinematic Feature Engineering and 12-Channel Representation	22
4.2.3	Phase-Aware Variant and Auxiliary Multi-Task Learning . . .	23
4.2.4	Temporal Interpolation and Spatial Anchoring	25
4.3	Pipeline Implementation	27
4.3.1	Video-Based Inference via MediaPipe	27
4.3.2	Feedback Visualization	27
4.4	Application Integration	28
4.4.1	Web Interface and Request Orchestration	28
4.4.2	Data Model and Persistence (Entity Framework Core)	29
4.4.3	Authentication and MVC Architecture	32
4.4.4	Error Handling and Robustness	34
5	Experiments and Results	35
5.1	Experimental Setup and Protocol	35
5.2	Quantitative Analysis	35
5.2.1	Dataset Characteristics	37
5.2.2	Stream Importance and Ablation Studies	38
5.3	Qualitative Analysis	39
5.3.1	End-to-End Clinical Feedback Walkthrough	39
5.3.2	Longitudinal Patient Tracking and Analytics	41
5.4	Generalization Challenges and Limitations	43
5.4.1	Error Analysis and Known Failure Modes	43
6	Conclusions and Future Directions	45
6.1	Thesis Contributions Summary	45
6.2	Key Empirical Findings	46
6.3	Limitations and Honest Assessment	46
6.4	Future Research Directions	47
6.5	Concluding Remarks	47
	Bibliography	48

Chapter 1

Introduction

1.1 Motivation

Physical rehabilitation is paramount for recovering functional mobility following surgery, injury, or neurological events like stroke. However, the majority of rehabilitation protocols require patients to complete routines independently at home. Without direct clinical supervision, patients frequently execute movements incorrectly, leading to suboptimal muscle engagement and an increased risk of secondary injuries. Concurrently, rapid advancements in accessible sensing technologies—such as RGB-based pose estimation and affordable depth cameras—have made the deployment of automated virtual coaching systems increasingly viable. Despite these technological strides, existing systems often fail to provide actionable, clinically relevant feedback. This thesis aims to bridge this gap by developing a system that delivers anatomically meaningful guidance, pinpointing precisely where and how movement quality degrades, rather than merely offering generic pass/fail assessments.

1.2 Problem Statement

Automated rehabilitation assessment presents a fundamentally different computational challenge compared to generic human action recognition. While action recognition asks *what* action is being performed, rehabilitation assessment must determine *how well* that specific action is executed. Most existing methodologies simplify this nuanced challenge into a binary classification problem, which fundamentally fails to capture the gradual, incremental progress characteristic of patient recovery.

Furthermore, machine learning models trained on relatively small-scale rehabilitation datasets are highly susceptible to “identity leakage.” In these instances, networks inadvertently memorize subject-specific physical characteristics (such as body pro-

portions) rather than learning the generalized kinematic patterns of the movement itself. Consequently, these models suffer from severe performance degradation when evaluated on unseen patients. Finally, the “black box” nature of traditional deep learning prevents clinicians from understanding *why* a movement was flagged as incorrect, severely limiting the clinical trust and adoption of these tools.

1.3 Aims and Objectives

The primary aim of this thesis is to design and develop an interpretable, subject-invariant deep learning framework for the automated quality assessment of physical rehabilitation exercises.

To achieve this overarching goal, the research pursues the following specific objectives:

- **Develop robust geometric normalization strategies** to mitigate subject-specific anthropometric bias, directly addressing the challenge of identity leakage and enabling strict cross-subject generalization (e.g., Leave-One-Subject-Out evaluation).
- **Design a modular feature extraction pipeline** that seamlessly transitions between baseline spatial coordinates and angle-augmented kinematic representations to capture complex temporal dynamics.
- **Formulate a contrastive learning paradigm**, utilizing validation-calibrated thresholding, to evaluate human movement quality on a continuous, quantifiable scale (ranging from 0 to 1), enabling the tracking of gradual patient recovery.
- **Generate Explainable AI (XAI) visual feedback** by mapping network attention weights back to the skeletal structure, actively highlighting the “Joints of Attention” (JoA) to localize biomechanical errors for the user.

Chapter 2

State of the Art and Theoretical Background

This chapter is organized into two parts. The first develops the theoretical foundations required to understand KinetoCheck, and the second reviews the most relevant prior work before synthesizing the research gaps that motivate the thesis contribution.

2.1 Theoretical Foundations

The following subsections introduce the conceptual building blocks used throughout the thesis.

2.1.1 Motion Analysis and Human Pose Estimation

Automated assessment of rehabilitation movements depends on reliable pose acquisition and clinically meaningful temporal interpretation. Two broad families dominate current systems: marker-based motion capture and markerless visual pose estimation.

Marker-based systems, such as Vicon, estimate body kinematics from reflective markers tracked by calibrated cameras, and are generally considered high-precision in controlled laboratory settings [ZGAA25]. Their primary strengths are geometric accuracy and repeatability; however, hardware cost, setup complexity, and the need for supervised capture environments reduce accessibility for home rehabilitation scenarios [ZGAA25].

Markerless systems, including MediaPipe Pose Landmarker, infer keypoints directly from RGB input and thus support low-cost, scalable deployment [LTN⁺19, BGR⁺20, CSWS17]. These approaches improve practicality, but are more sensitive to occlusions, illumination variability, camera angle changes, and partial-body visibility

[BGR⁺20, CSWS17]. Consequently, a practical rehabilitation system must treat pose extraction uncertainty as a first-class design constraint.

For this thesis, the training phase relies on the UI-PRMD dataset, while deployment relies on RGB-based pose extraction. This cross-domain setup requires representation-level compatibility between high-quality motion capture trajectories and noisier in-the-wild visual keypoints. Theoretical implications include the need for invariant normalization, stable temporal resampling, and robustness-oriented feature design.

The dataset composition used in this thesis is summarized in Table 5.4 (see Chapter 5 for full dataset details used in experiments). 3D joint visualizations for the original Vicon and the reduced 17-joint mapping are shown in Chapter 5 (Figure 5.2).

2.1.2 Skeleton-Based Learning for Time Series

Human movement can be modeled as a time-indexed sequence of articulated joint states. In this formulation, each frame is represented by a set of keypoints, and a complete repetition forms a multivariate temporal signal.

Let T denote the number of time steps, N the number of joints, and C the coordinate dimension. A skeletal sequence is represented as:

$$X \in \mathbb{R}^{T \times N \times C} \quad (2.1)$$

In rehabilitation assessment, this representation is preferred over raw RGB because it suppresses texture/background noise and emphasizes biomechanical structure [ZGAA25].

Graph modeling is a natural extension of this formulation. The body is encoded as a graph $G = (V, E)$, where nodes correspond to joints and edges to anatomical relations. This captures the non-Euclidean dependencies among body parts and enables locality-aware processing across physically connected joints [CXJS22, GYDD18]. Compared with flat vector encodings, graph representations preserve meaningful topology needed to identify compensatory motion patterns.

In KinetoCheck, additional temporal features such as velocity and acceleration are used to represent execution quality beyond static posture. The theoretical motivation is that incorrect movement often manifests as trajectory irregularity or unstable timing rather than endpoint error alone.

The joint mapping used for representation alignment is summarized in Table 2.1.

Vicon Marker	Index	Target Joint	Target Index	Mapping Rule	Included
LFHD	1	nose	0	midpoint(1,2)	Y
RFHD	2	nose	0	midpoint(1,2)	Y
LBHD	3			not.used	N
RBHD	4			not.used	N
C7	5			not.used	N
T10	6			not.used	N
CLAV	7			not.used	N
STRN	8			not.used	N
RBAK	9			not.used	N
LSHO	10	left.shoulder	1	direct	Y
LUPA	11			not.used	N
LELB	12	left.elbow	3	direct	Y
LFRM	13			not.used	N
LWRA	14	left.wrist	5	midpoint(14,15)	Y
LWRB	15	left.wrist	5	midpoint(14,15)	Y
LFIN	16			not.used	N
RSHO	17	right.shoulder	2	direct	Y
RUPA	18			not.used	N
RELB	19	right.elbow	4	direct	Y
RFRM	20			not.used	N
RWRA	21	right.wrist	6	midpoint(21,22)	Y
RWRB	22	right.wrist	6	midpoint(21,22)	Y
RFIN	23			not.used	N
LASI	24	left.hip	7	midpoint(24,26)	Y
RASI	25	right.hip	8	midpoint(25,27)	Y
LPSI	26	left.hip	7	midpoint(24,26)	Y
RPSI	27	right.hip	8	midpoint(25,27)	Y
LTHI	28			not.used	N
LKNE	29	left.knee	9	direct	Y
LTIB	30			not.used	N
LANK	31	left.ankle	11	direct	Y
LHEE	32	left.heel	13	direct	Y
LTOE	33	left.foot_index	15	direct	Y
RTHI	34			not.used	N
RKNE	35	right.knee	10	direct	Y
RTIB	36			not.used	N
RANK	37	right.ankle	12	direct	Y
RHEE	38	right.heel	14	direct	Y
RTOE	39	right.foot_index	16	direct	Y

Table 2.1: Joint mapping from Vicon markers to 17-joint model

2.1.3 Graph Neural Networks and ST-GAT for Rehabilitation Assessment

Graph Neural Networks (GNNs) extend deep learning to structured relational data. For movement analysis, spatial-temporal graph models process inter-joint relations and time evolution, which makes them suitable for quality-sensitive exercise assessment [CXJS22, GYDD18].

Spatio-Temporal Graph Attention Networks (ST-GAT) improve over fixed-neighborhood graph convolutions by learning attention weights that adapt to exercise phase and local error context. Intuitively, not all joints are equally informative at every moment. During one phase of an exercise, knee and hip joints may dominate quality evidence, while during another phase trunk compensation may become decisive.

Formally, attention coefficients are computed from learned node projections and normalized over local neighborhoods. This mechanism gives the model adaptive selectivity while retaining graph structure. Graph attention with multi-scale temporal modeling further increases representational diversity by allowing parallel focus patterns [CXJS22].

Temporal modeling can be integrated with recurrent models (RNN/LSTM), temporal convolutional networks (TCN), or transformer-style self-attention. RNNs capture sequential context but may struggle with long dependencies and parallelization. TCNs are efficient and stable for fixed-window processing. Transformers model long-range relations but often require larger datasets and careful regularization [CXJS22, GYDD18]. A temporal pyramid strategy offers a practical compromise: multiple temporal resolutions improve robustness to speed variation while maintaining manageable computational complexity.

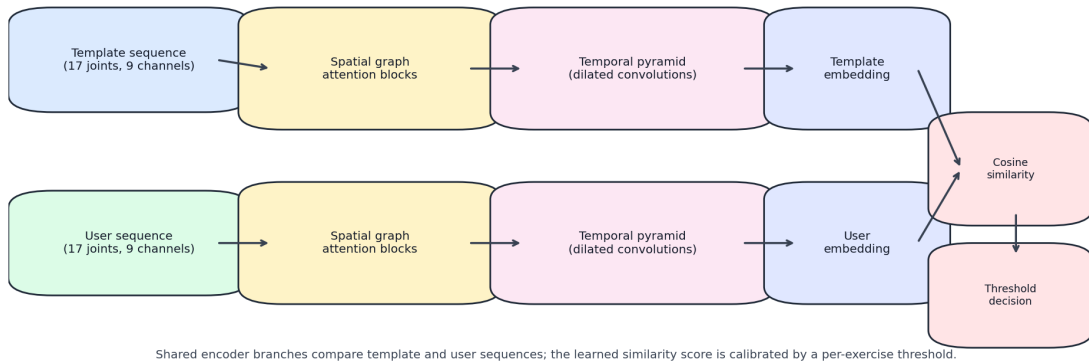


Figure 2.1: Siamese ST-GAT with temporal pyramid architecture used in the thesis.

2.1.4 Siamese Networks and Pairwise Metric Learning

Usual classification predicts discrete labels. In rehabilitation, however, movement quality is often continuous and progression-oriented. Pairwise metric learning addresses this by embedding repetitions into a latent space where geometric distance reflects execution similarity [BGL⁺93, HCL06, KZS15].

Given two sequences x_i and x_j , a Siamese encoder $f(\cdot)$ produces embeddings $z_i = f(x_i)$ and $z_j = f(x_j)$. A similarity score is then derived from a distance or similarity function such as cosine similarity. Contrastive objectives enforce smaller distances for similar-quality pairs and larger distances for dissimilar-quality pairs.

For the contrastive formulation used in this thesis, the model learns from pairs rather than triplets. Let z_a and z_b be the embeddings of a template and a user execution, and let $y \in \{0, 1\}$ indicate whether the pair is similar. A standard objective is:

$$\mathcal{L}_{\text{contrastive}} = y d(z_a, z_b)^2 + (1 - y) \max(0, m - d(z_a, z_b))^2 \quad (2.2)$$

where m is a positive margin. Decision thresholds can then convert similarity scores into correct/incorrect predictions, while preserving the option to report confidence-like or quality-like scalar outputs.

From a theoretical perspective, this paradigm is aligned with rehabilitation goals: it supports both binary decisions for screening and graded progress tracking across sessions.

2.1.5 Explainability in Movement Assessment

Clinical usability requires not only a decision but also an explanation. Attention-based models provide one interpretable signal: joint-level importance derived from learned attention patterns. In KinetoCheck, this notion is operationalized as problematic or salient joints that contribute strongly to a low-quality decision.

Explainability in this context should be treated cautiously. High attention does not automatically imply causal biomechanical error, and visual explanations may be affected by input noise, landmark jitter, and graph connectivity assumptions [JW19, AGM⁺18, Rud19]. Therefore, attention-based feedback is best interpreted as decision support, not as a substitute for expert diagnosis.

Actionable feedback design must satisfy three conditions: localizability (which joints), temporal context (when in the motion cycle), and readability (how users understand corrective hints). These principles motivate coupling model outputs with visual overlays and structured textual summaries in the practical system.



Figure 2.2: Example Annotated Video with 2 skeleton overlays, blue for the template skeleton and white/yellow/red for the mediapipe skeleton with corresponding colors for the worst joints.

2.2 Review of Related Work

This section surveys the most relevant movement-assessment literature and positions the thesis against it.

2.2.1 Traditional Movement Assessment

Early systems for rehabilitation assessment relied on non-learning-based techniques, including Dynamic Time Warping (DTW) and Gaussian Mixture Models (GMM). While these methods were interpretable, they struggled with subject variability and lacked the capacity to learn deep hierarchical movement representations. Comparative benchmark analyses on rehabilitation datasets further motivate the transition to

more expressive learned representations [ZGAA25].

2.2.2 The Pioneer Deep Learning Framework (Liao et al.)

The first significant deep-learning implementation for rehabilitation assessment was proposed by Liao et al. (2020), who introduced the UI-PRMD dataset and established a benchmark. The model introduced a Temporal Pyramid + LSTM design, where parallel 1D-CNN branches processed joint displacement signals at different time scales before temporal integration with a multi-layer LSTM. This foundational work demonstrated that hierarchical spatio-temporal modeling is essential for movement quality regression. It also highlighted the importance of multi-scale temporal representation, a principle adopted in the ST-GAT architecture used in this thesis.

2.2.3 Contemporary GCN and Transformer Models

Recent research has moved toward graph-based architectures that preserve human body topology more effectively [CXJS22, GYDD18]. Notable contemporary approaches include **ExeChecker (2024–2025)**, which introduced Joints of Attention (JoA) by extracting attention weights from STGAT backbones to provide localized feedback on problematic joints, directly influencing KinetoCheck’s interpretability design. **D-STGCNT** combines dense graph convolutions with GRU and Transformer components for high-accuracy quality scoring with efficient inference, exemplifying transformer-augmented spatial-temporal graph modeling [CXJS22]. **EGCN++** represents strong state-of-the-art performance on UI-PRMD by fusing position and orientation features. More broadly, recent analyses show that feature design and model choice remain strongly dataset-dependent [ZGAA25].

2.2.4 Video-Based and Smartphone Telerehabilitation

Recent implementations by Chander et al. (2025) transitioned these methods to RGB-only settings using MediaPipe keypoint extraction. Their results highlight the feasibility of home deployment through live repetition counting via autocorrelation and lightweight Transformer-based recognition. This work directly motivates the deployment choices made in KinetoCheck.

2.3 Critical Synthesis and Research Gaps

Despite substantial progress in the field, three critical gaps remain unresolved. First, cross-domain consistency between laboratory motion capture and RGB-based in-

ference remains underexplored; this thesis addresses it through systematic representation alignment. Second, many high-performing methods emphasize accuracy while providing limited clinically actionable interpretability; KinetoCheck prioritizes joint-level feedback mechanisms. Third, thresholding and score calibration strategies are often underreported, which reduces reproducibility and practical comparability across studies [ZGAA25, CXJS22].

By combining representation alignment, Siamese ST-GAT similarity learning, and interpretable joint-level feedback under strict subject-invariant evaluation protocols, this thesis contributes a comprehensive framework that addresses these gaps. The next chapter formalizes the practical system requirements and architectural design needed to realize this framework as an operational application.

Chapter 3

Application Requirements and System Architecture

This chapter connects the theoretical foundations from Chapter 2 with the actual implementation of KinetoCheck. It begins by turning the broad research goals and identified gaps into a concrete set of system requirements. From there, the modular architecture is introduced, breaking down the layered components and software design patterns that bring the platform to life. Together, this requirements-and-architecture blueprint sets the stage for the practical coding details and experimental results that follow.

3.1 System Scope and Problem Definition

KinetoCheck addresses the practical problem of evaluating rehabilitation exercise quality from user-provided videos, with automated feedback that is both machine-generated and interpretable. The application is designed for a semi-supervised usage scenario in which a user uploads a recording, the pipeline performs pose extraction and model inference, and the system returns a decision, score, and explanatory artifacts.

3.2 System Requirements

The system requirements are divided into functional and non-functional concerns so that the application goals and quality constraints remain explicit.

3.2.1 Functional Requirements

Functional requirements define what the application must do.

1. **Video Upload:** The system shall accept a user-uploaded exercise video in supported formats: mp4, mov, mkv, avi, and webm.
2. **Input Validation:** The system shall validate file type, file size, and basic integrity before processing.
3. **Pose Extraction:** The system shall run MediaPipe Pose Landmarker to extract keypoints frame-by-frame from the uploaded video.
4. **Preprocessing Consistency:** The extracted keypoints shall be transformed using the same preprocessing logic used during model training.
5. **Inference Execution:** The system shall load the exercise-specific checkpoint and compute a similarity-based prediction.
6. **Decision and Score Reporting:** The system shall return a predicted label (correct/incorrect), a score, and the threshold used for the decision.
7. **Joint-Level Feedback:** The system shall identify and expose high-importance/problematic joints used for feedback visualization.
8. **Annotated Output:** The system shall produce an annotated video or frame overlays that highlight key feedback regions.
9. **Structured Report Export:** The system shall generate and store a JSON report containing core inference artifacts.
10. **Analytics Summary:** The system shall generate user-level exercise statistics and overview metrics from inference outputs.
11. **Asset Retrieval:** The web layer shall provide access to generated reports and media artifacts.

3.2.2 Non-Functional Requirements

Non-functional requirements define quality attributes and operational constraints.

1. **Reproducibility:** Given the same input and model version, the system should produce stable outputs within acceptable numerical tolerance.
2. **Maintainability:** Training, inference, and web modules should remain separated to support independent updates.
3. **Latency Expectation:** End-to-end processing time should be bounded for practical usability, with an acceptance target of processing time $\leq 1.0 \times$ input video duration on target deployment hardware.

4. **Robustness to Missing Detections:** The pipeline should handle partial landmark dropout through interpolation, masking, or safe fallback behavior.
5. **Browser Compatibility:** The web layer targets the latest two stable versions of Chrome, Edge, Firefox, and Safari.
6. **Traceability:** Each result should be linked to model version, threshold configuration, and processing timestamp.
7. **Error Transparency:** Invalid inputs or failed processing stages should produce explicit and user-readable error messages.
8. **Analytics Consistency:** Aggregated statistics should remain traceable to the underlying inference session, model version, and timestamp.

The consolidated functional and non-functional requirements matrix is presented in Table 3.1.

The requirements above define the boundaries for the implementation. The following architectural overview shows how the application components map onto those requirements.

3.3 Architectural Overview

KinetoCheck is organized as a modular pipeline that separates input acquisition, preprocessing, model inference, feedback generation, and reporting. This separation keeps the learning components independent from the application layer so that model variants can be evaluated without changing the user workflow.

Table 3.1: T5 Functional and non-functional requirements matrix

Req ID	Type	Requirement	Priority	Verification Method	Acceptance Criterion
F-01	F	User shall upload a supported video file for analysis.	Must	Functional test	Valid file is accepted and processing starts.
F-02	F	System shall extract 17-joint pose sequence from uploaded video.	Must	Pipeline log + output check	Sequence tensor generated with expected shape.
F-03	F	System shall run checkpoint-based inference and output label/score.	Must	Functional test	Report contains predicted label, score, threshold.
F-04	F	System shall generate an annotated output video.	Must	Artifact inspection	Annotated MP4 exists and is downloadable.
F-05	F	System shall provide problematic-joint feedback in report.	Should	JSON schema check	worst.joints field includes ranked entries.
NF-01	NF	Pipeline shall be reproducible from scripts/configuration.	Must	Re-run reproducibility test	Comparable metrics reproduced from same checkpoint/data.
NF-02	NF	System shall be maintainable with modular preprocessing/model/inference code.	Should	Code review	Clear module boundaries and documented interfaces.
NF-03	NF	Inference shall tolerate temporary landmark dropouts.	Must	Robustness test	No crash; missing frames handled with fallback strategy.
NF-04	NF	Web output shall be browser-compatible for video playback.	Should	Cross-browser test	Playable or downloadable output in major browsers.

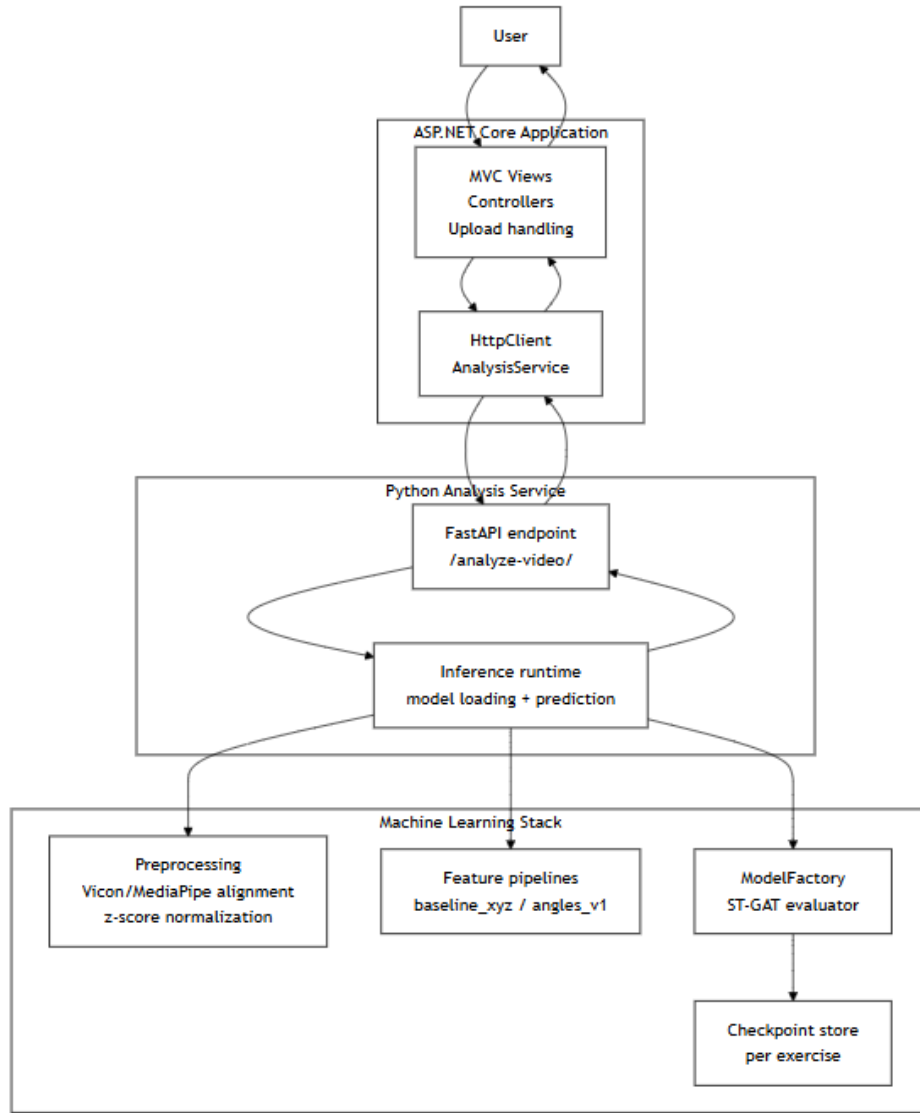


Figure 3.1: System Context Diagram depicting the separation of concerns between the presentation (ASP.NET), API (FastAPI), and machine learning layers.

The system can be summarized in five layers:

1. **Input layer:** receives exercise videos, live camera streams, or dataset sequences.
2. **Preprocessing layer:** extracts 17-joint pose sequences, aligns dataset formats, normalizes body coordinates, and constructs feature tensors.
3. **Inference layer:** applies the ST-GAT encoder, optional phase alignment, optional frame-level decoding, and similarity scoring.
4. **Output layer:** produces predictions, score values, and joint-level feedback for visualization.
5. **Analytics layer:** aggregates repeated inference sessions into user-level statistics, exercise summaries, and trend reports.

3.3.1 Model and Inference Design

The data pipeline must preserve compatibility between training and inference domains. Input video is transformed into a temporal sequence of 17-joint keypoints. The fixed tensor specification is: raw pose sequence as float32 with shape $(N, 17, 3)$, preprocessed sequence as float32 with shape $(120, 17, 3)$, and model input as float32 with shape $(B, C, 120, 17)$.

The final response package includes: prediction label and score; threshold and model identifier; problematic-joint list; annotated output media path; and JSON report path.

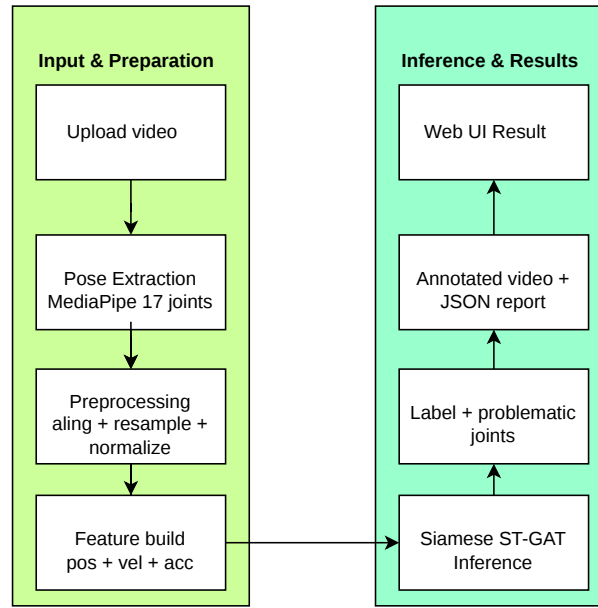


Figure 3.2: End-to-end input-output flow of the KinetoCheck application, from video upload to generated report and annotated media.

3.3.2 Data Flow and Artifact Specification

Prediction is derived from a similarity score compared against an exercise-specific threshold. The thresholding policy must be versioned because it directly influences precision-recall behavior and user feedback interpretation. In the current implementation, threshold calibration sweeps 101 linearly spaced threshold candidates, selects the threshold that maximizes validation F1, and applies the decision rule *correct* if $\text{score} \geq \text{threshold}$.

To ensure reproducibility, the practical system documents the following elements:

- train/validation/test partition policy (including LOSO if used);

- hyperparameter configuration and optimizer settings;
- checkpoint naming/versioning convention;
- per-exercise metrics and threshold logs;
- software dependencies and runtime environment.

3.3.3 Analytics and Reporting Layer

A dedicated analytics and reporting layer sits after inference and before long-term user-facing outputs. The purpose of this layer is to convert individual inference results into a longitudinal view of rehabilitation progress.

The analytics pipeline reuses existing inference artifacts instead of re-running the model. A typical session flow is:

1. run exercise inference and store the score, threshold, label, and joint feedback;
2. attach metadata such as exercise identifier, model version, and timestamp;
3. aggregate repeated sessions by user, exercise, or time window;
4. compute summary statistics and trend values;
5. deliver the result to the dashboard or export it as JSON or CSV.

3.4 Software Design Patterns

3.4.1 Factory and Singleton Cache Patterns

The factory pattern is used to create the correct evaluator implementation based on the selected checkpoint configuration. Loaded models are cached so that repeated inference requests do not reload weights from disk, reducing startup overhead.

3.4.2 Composition and Component Isolation

The phase aligner, frame decoder, and joint scorer are implemented as separate components. This keeps the architecture testable and makes it possible to improve one part of the system without rewriting the entire model stack.

This chapter has formalized both the practical requirements and the architectural design of KinetoCheck. By bridging specification and implementation design, it establishes the engineering baseline needed to interpret the training strategies and implementation details presented in the next chapter.

Chapter 4

Training Design and Software Implementation

This chapter details how the architectural requirements specified in Chapter 3 are realized through training strategies and software implementation. It first presents the training design, including the metric learning objective and the evaluation framework used to validate strict generalization. Subsequently, it describes the implementation details of key system components.

4.1 Model Training Strategy

The training implementation is organized around a reusable trainer and three thin entry scripts. The baseline script uses a fixed XYZ-based feature pipeline, the factory-based script allows feature extraction strategies to be selected, and the phase-aware script enables auxiliary correction outputs and ROM regularization.

4.1.1 Spatio-Temporal Graph Attention Network (ST-GAT) Architecture

The core processing engine of the KinetoCheck framework is the Spatio-Temporal Graph Attention Network (ST-GAT). Unlike traditional Convolutional Neural Networks (CNNs) that process dense pixel grids, or standard Recurrent Neural Networks (RNNs) that process flattened coordinate arrays, the ST-GAT preserves the explicit geometric and anatomical structure of the human body.

Building upon the theoretical graph representation established in Section 2.1.2, KinetoCheck implements the spatio-temporal graph $\mathcal{G}$ utilizing the 17-joint Mediapipe topology. Rather than passing raw coordinates, each node $\mathcal{V}$ is populated with the comprehensive 12-channel kinematic feature vector detailed in Section 4.2.2. The

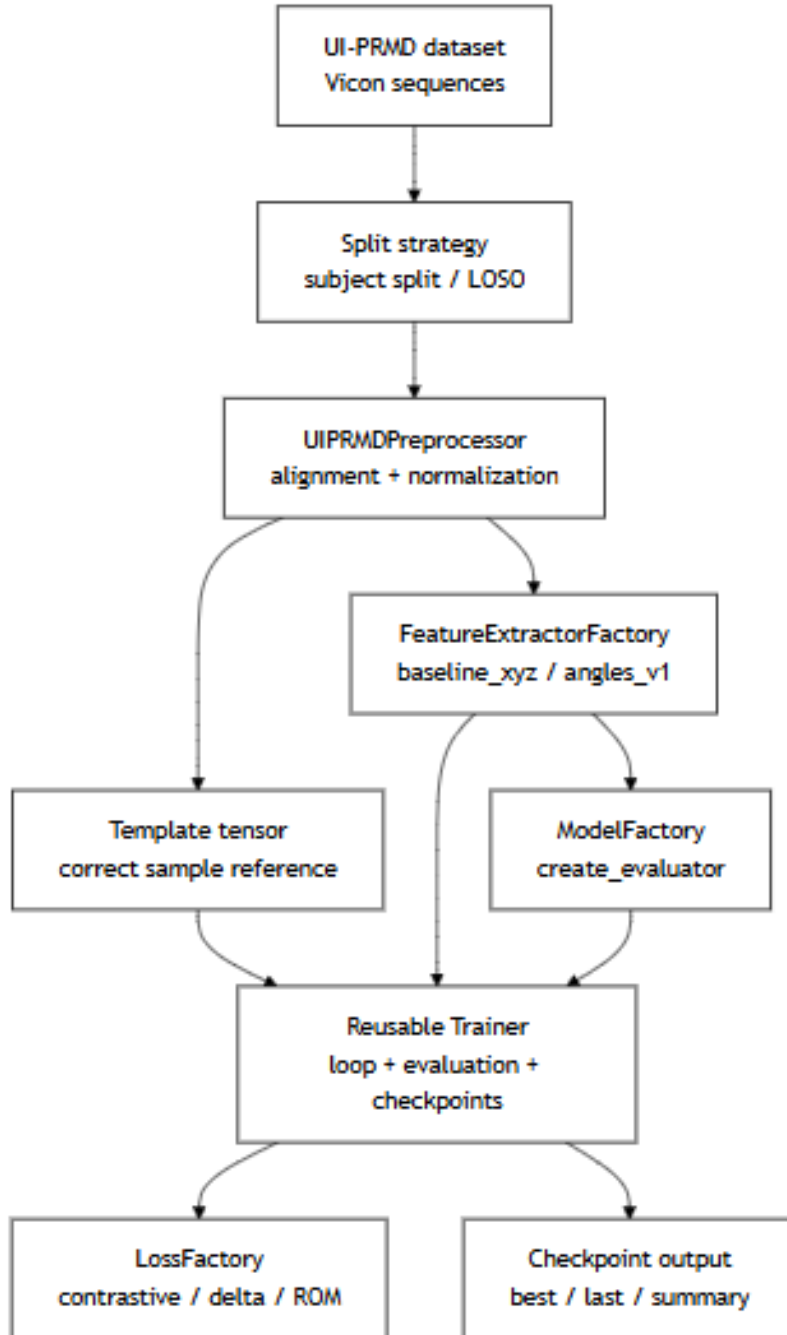


Figure 4.1: The modular training architecture, highlighting the integration of the reusable trainer with feature and loss factories.

edges $\mathcal{E}$ are strictly defined by the natural anatomical connections of the human skeleton, forming the spatial adjacency matrix used during the forward pass.

Spatial Graph Attention

In the spatial domain, the network must understand how the movement of one joint affects its neighbors (e.g., how a collapsed knee affects the hip). Graph Attention Layers (GAT) are utilized to compute dynamic edge weights. During the forward pass, the model calculates an attention coefficient $\alpha_{i,j}$ for every pair of connected joints i and j . This coefficient represents the biomechanical importance of joint j 's features to joint i 's current state.

This attention mechanism is the foundation of the platform's Explainable AI (XAI) capabilities. Rather than remaining a "black box," the pooled spatial attention weights are explicitly extracted during inference and persisted into the `JointInsight` database table (as discussed in Section 4.4.2). These weights determine the `Importance` and `ProblemScore` metrics, allowing the frontend to highlight specific failing joints on the user's dashboard.

Temporal Pyramids

Human movement occurs across multiple temporal scales simultaneously; a localized tremor might last only a few frames, while the overall repetition spans the entire sequence. To capture this without the gradient instability often associated with LSTMs, the ST-GAT implements 1D Temporal Convolutional Pyramids.

After the spatial attention layer processes the frame-wise geometry, the temporal module applies 1D dilated convolutions along the time dimension. By using parallel convolutional branches with varying receptive fields (e.g., kernel sizes of 3, 5, and 7), the network simultaneously captures high-frequency immediate anomalies (like sudden instability) and low-frequency long-term trends (like the overall speed of the repetition).

The outputs of the spatial and temporal layers are subsequently pooled into a final 128-dimensional latent embedding vector, which acts as the condensed representation of the entire exercise execution.

4.1.2 Mathematical Formulation of the Contrastive Objective

The baseline and factory-based variants optimize a contrastive loss over template and user embeddings. Rather than predicting a discrete class label, the Siamese architecture maps input sequences into a 128-dimensional latent space where the

Euclidean distance between embeddings represents the biomechanical similarity of the executions.

Let $f_\theta(\cdot)$ denote the ST-GAT encoder parameterized by weights θ . Given a template sequence x_t and a user sequence x_u , the network generates L_2 -normalized embeddings $z_t = f_\theta(x_t)$ and $z_u = f_\theta(x_u)$. A ground-truth label $y \in \{0, 1\}$ is provided, where $y = 1$ indicates that the user sequence is a correct execution (a positive pair), and $y = 0$ indicates an incorrect execution (a negative pair).

The optimization objective minimizes the contrastive loss function $\mathcal{L}$, defined as:

$$\mathcal{L}(z_t, z_u, y) = y \cdot \|z_t - z_u\|_2^2 + (1 - y) \cdot \max(0, m - \|z_t - z_u\|_2)^2 \quad (4.1)$$

Where:

- $\|z_t - z_u\|_2$ is the Euclidean distance between the template and user embeddings.
- m is the margin hyperparameter (set to 1.0 in this implementation).

During the forward pass, if the pair represents a correct execution ($y = 1$), the second term zeros out, and the network penalizes the squared distance, forcing the embeddings closer together. Conversely, if the pair is incorrect ($y = 0$), the network ignores the first term and applies a hinge loss. The margin m dictates that negative pairs are only penalized if their distance is less than 1.0. Once the embeddings are pushed sufficiently far apart, the loss becomes zero, preventing the network from aggressively separating already distinct representations and thereby stabilizing the gradient updates.

This produces a continuous similarity score that is later compared against a validation-derived threshold to produce the final binary classification.

Implementation of the Contrastive Loss

To bridge the theoretical formulation and the software architecture, the contrastive loss is implemented as a custom PyTorch module. The exact implementation used in the training pipeline is shown in Listing 4.1.

Listing 4.1: PyTorch implementation of the Contrastive Loss module.

```

1 import torch
2 import torch.nn as nn
3
4 class ContrastiveLoss(nn.Module):
5     def __init__(self, margin: float = 1.0):
6         super().__init__()
7         self.margin = margin

```

```

8
9     def forward(self, emb_a: torch.Tensor, emb_b: torch.Tensor,
10                labels: torch.Tensor) -> torch.Tensor:
11         # Compute pairwise Euclidean distance
12         distance = torch.norm(emb_a - emb_b, p=2, dim=1)
13
14         # Loss for correct pairs (labels == 1.0)
15         loss_correct = labels * torch.pow(distance, 2)
16
17         # Loss for incorrect pairs (labels == 0.0)
18         loss_incorrect = (1.0 - labels) * torch.pow(
19             torch.clamp(self.margin - distance, min=0.0), 2
20         )
21
22         # Return mean loss over the batch
23         return torch.mean(loss_correct + loss_incorrect)

```

As seen in Listing 4.1, the embeddings are processed in parallel batches. The `torch.norm` function computes the L_2 distance across the embedding dimension (`dim=1`). The `torch.clamp` function elegantly applies the hinge mechanism for incorrect pairs, ensuring that gradients are only propagated if the distance is smaller than the configured margin. This modular design allows the `LossFactory` to instantiate the loss function dynamically based on the chosen training configuration.

4.2 Data Processing and Normalization

4.2.1 Identity Leakage Mitigation

To reduce identity leakage, subject-wise split strategies are used so that no subject appears in more than one split. The codebase also includes explicit Leave-One-Subject-Out (LOSO) utilities for stricter generalization evaluation. In preprocessing, pose sequences are centered and normalized consistently so that training and inference use the same representation.

4.2.2 Kinematic Feature Engineering and 12-Channel Representation

Raw spatial coordinates are often insufficient to fully capture the dynamic nature of physical rehabilitation exercises. To provide the ST-GAT model with a comprehensive

understanding of movement biomechanics, the preprocessed 3D coordinates are expanded into a 12-channel feature representation prior to inference. For each joint at every time step, the following kinematic features are computed:

- **Channels 1–3 (Spatial Position):** The normalized X, Y, and Z coordinates of the joint.
- **Channels 4–6 (Velocity):** The first-order temporal derivative of the position, representing the instantaneous speed and direction of the joint.
- **Channels 7–9 (Acceleration):** The second-order temporal derivative of the position, indicating changes in movement speed and highlighting jitter or instability.
- **Channel 10 (Joint Angle):** The angle formed between anatomically connected adjacent joints (e.g., the angle at the elbow formed by the shoulder and the wrist).
- **Channel 11 (Angular Velocity):** The temporal derivative of the joint angle, capturing the speed of rotation during the execution of the movement.
- **Channel 12 (Normalized Bone Length):** The distance between connected joints, normalized by the length of the torso (computed as the distance between the left hip and the left shoulder).

By incorporating velocity and acceleration alongside static position, the model successfully captures trajectory irregularities and unstable timing, which are primary indicators of poor execution quality. Furthermore, the inclusion of normalized bone ratios ensures that the resulting feature space remains robust against natural variations in subject height, body proportions, and camera distance.

4.2.3 Phase-Aware Variant and Auxiliary Multi-Task Learning

While the baseline ST-GAT encoder effectively classifies overall movement quality, clinical rehabilitation requires granular feedback regarding partial-range executions and localized joint errors. To address this, the phase-aware variant transforms the network into a multi-task learning architecture by appending two auxiliary prediction heads to the latent embedding: a Range of Motion (ROM) regularizer and a Frame Decoder.

Range of Motion (ROM) Regularization

Patients often compensate for muscle weakness by terminating a movement prematurely. The ROM penalty continuously measures the maximum spatial displacement (amplitude) of the user’s joints and compares it against the expected amplitude of the expert template. If a user sequence is labeled as incorrect due to a truncated range of motion, the ROM penalty applies a secondary gradient update, explicitly teaching the spatial attention layers to penalize incomplete joint trajectories.

Frame-Level Error Decoding

To further enhance the Explainable AI (XAI) capabilities, a Multi-Layer Perceptron (MLP) Frame Decoder is attached to the user embedding. This decoder projects the 128-dimensional latent representation into a 17-dimensional vector, attempting to explicitly predict the localized spatial error magnitude at each joint. This is optimized using a Mean Squared Error (MSE) loss against the ground-truth spatial delta (ΔX) between the user and the aligned template.

Composite Loss Formulation

The total multi-task loss $\mathcal{L}_{total}$ is calculated as a weighted sum of the primary contrastive objective and the two auxiliary penalties:

$$\mathcal{L}_{total} = \mathcal{L}_{contrastive} + \lambda_{rom} \cdot \mathcal{L}_{ROM} + \lambda_{delta} \cdot \mathcal{L}_{delta} \quad (4.2)$$

Where λ_{rom} and λ_{delta} are the configurable hyperparameter weights assigned to the auxiliary objectives (detailed in Table 4.1).

Benefits of this phase-aware multi-task design include:

- **Targeted Supervision:** The auxiliary gradients force the network to learn correction-aware spatial representations, improving the accuracy of the extracted attention weights.
- **Reduced False Positives:** ROM regularization prevents the model from accidentally validating partial-range executions that otherwise maintain correct anatomical posture.
- **Rich Metadata:** The decoder outputs are persisted directly into the checkpoint, enabling the advanced joint-level feedback extraction (the `JointInsight` entity) utilized during inference.

Table 4.1 summarizes the key hyperparameters and training choices used for the reported experiments. For reproducibility, per-fold checkpoints are saved, and the training and validation splits used for each fold are stored.

Group	Parameter	Value	Unit/Range	Comment
model	input channels	9/12	channels	xyz (+ velocity + acceleration)
model	hidden channels	(64, 128)	tuple	ST-GAT blocks
model	embedding dim	128	dim	Siamese projection head
model	use phase decoder	True	boolean	Enables temporal alignment module
optimizer	learning rate	1e-3	-	Initial learning rate
optimizer	weight decay	1e-4	-	L2 regularization penalty
optimizer	margin	1.0	-	Contrastive loss margin
training	delta weight	0.0	-	Frame-level correction loss weight
training	rom weight	2.0	-	Range-of-motion loss weight
data	batch size	8	samples	Training batch size
data	num workers	0	threads	DataLoader workers
training	epochs	30	epochs	Maximum training epochs
training	patience	10	epochs	Early stopping patience
training	seed	42	-	Random seed for reproducibility
training	device	auto	-	Hardware accelerator
split	split mode	subject	-	Determines data partitioning strategy
split	train/val/test ratio	0.8/0.1/0.1	ratio	Dataset split proportions

Table 4.1: Model hyperparameters and training settings

4.2.4 Temporal Interpolation and Spatial Anchoring

A fundamental challenge in automated movement assessment is temporal and spatial misalignment. A patient recovering from an injury will inherently execute a repetition at a different speed and possess different anatomical proportions compared to the healthy expert template. If the system were to compute error margins on a rigid, unaligned basis, a perfectly executed but slower movement would be severely penalized.

To resolve this and generate an intuitive visual overlay (the “ghost” skeleton), KinetoCheck implements a deterministic, dual-stage alignment module consisting of Linear Temporal Interpolation and Spatial Anchoring.

Linear Temporal Sequence Interpolation

Given a user kinematic sequence of length T_u and a template sequence of length T_t , the system maps the discrete time indices of the user’s execution into the continuous temporal space of the template. For each user frame $i \in \{0, \dots, T_u - 1\}$, the

corresponding fractional template index t_i is computed via linear spacing:

$$t_i = i \cdot \frac{T_t - 1}{T_u - 1} \quad (4.3)$$

Because t_i is a continuous floating-point value, the exact template posture must be approximated using 1D linear interpolation. By isolating the lower bound index $idx_0 = \lfloor t_i \rfloor$ and the upper bound index $idx_1 = \min(idx_0 + 1, T_t - 1)$, the fractional remainder $\alpha = t_i - idx_0$ acts as a blending weight. The temporally aligned template frame $\hat{X}_t^{(i)}$ is thus computed as a weighted combination of two adjacent expert frames:

$$\hat{X}_t^{(i)} = (1 - \alpha) \cdot X_t^{(idx_0)} + \alpha \cdot X_t^{(idx_1)} \quad (4.4)$$

This continuous temporal scaling ensures the ghost skeleton remains perfectly synchronized with the user's kinematic progression throughout the entire duration of the exercise, regardless of execution speed.

Spatial Anchoring and Proportional Scaling

Once temporally synchronized, the template must be spatially normalized to match the user's specific body metrics and camera distance. The system calculates a global scaling factor S , derived from the median vertical distance between the distal anatomical points (the nose and the feet) across both sequences:

$$S = \max \left(0.5, \min \left(4.0, \frac{\text{median}(\text{Height}_{user})}{\text{median}(\text{Height}_{template})} \right) \right) \quad (4.5)$$

Finally, a biomechanical fulcrum (such as the midpoint between the left and right hips, or the ankles) is selected as a translation anchor A . The temporally interpolated template frame $\hat{X}_t^{(i)}$ is centered around its own anchor, scaled by the factor S , and translated to match the user's spatial anchor coordinates in the 2D plane:

$$\text{Ghost}^{(i)} = \left(\hat{X}_t^{(i)} - A_{template}^{(i)} \right) \cdot S + A_{user}^{(i)} \quad (4.6)$$

This combined spatio-temporal alignment allows the rendering engine to project a highly accurate, anatomically proportional, and time-locked corrective overlay directly onto the user's video feed.

4.3 Pipeline Implementation

4.3.1 Video-Based Inference via MediaPipe

For RGB input, the system uses MediaPipe Pose Landmarker to extract frame-wise 17-joint landmarks. The model infers 33 landmarks per frame; these are remapped to the 17-joint representation used during training via a deterministic mapping defined in the preprocessing layer.

4.3.2 Feedback Visualization

To expose model reasoning, the visualization module overlays the extracted skeleton on the input video and highlights the joints that are most relevant to the model output. This produces intuitive corrective cues for users and therapists.

At startup, the inference module caches loaded checkpoints in memory to avoid repeated disk I/O. Given a preprocessed pose sequence of shape $(1, C, 120, 17)$, the inference function: (1) builds the selected feature representation; (2) passes the tensor through the evaluator; (3) computes a similarity score; (4) compares the score against the learned threshold to produce a binary decision; and (5) returns a structured result object.

Spatio-Temporal Rendering and the Ghost Skeleton Overlay

To maximize the clinical interpretability of the system, a “ghost skeleton” overlay was implemented. This visual artifact superimposes a semi-transparent representation of the optimal template movement directly onto the user’s video feed.

The geometric scaling and frame-synchronization of this overlay are handled by the deterministic alignment module detailed previously in 4.2.4. However, applying this math to in-the-wild videos introduces a final rendering challenge: **Dynamic Camera Orientation Detection**. Because home rehabilitation videos exhibit varying camera angles, hardcoding the lateral axis often leads to distorted 2D projections. The rendering engine automatically resolves this by measuring the shoulder separation across the X and Y dimensions, selecting the axis with the maximum geometric variance to correctly orient the template before applying the spatial anchoring equations.

This advanced rendering pipeline ensures that patients receive real-time, proportionally accurate visual cues, clearly illustrating the spatial delta between their current posture and the idealized target movement without suffering from temporal desynchronization.

4.4 Application Integration

The ASP.NET Core web interface orchestrates the request lifecycle: upload, validation, forwarding of the video to the Python analysis service, and response rendering.

4.4.1 Web Interface and Request Orchestration

1. **Upload:** User selects a video file; the interface validates MIME type and file size.
2. **Temporary Storage:** The video is written to a temporary directory with a UUID-based filename.
3. **Pose Extraction:** MediaPipe is invoked; keypoints are logged.
4. **Preprocessing:** Keypoints are normalized and resampled.
5. **Inference:** The pre-trained model is loaded and a prediction is computed.
6. **Visualization:** An annotated video is generated.
7. **Report Generation:** A JSON report is written containing core artifacts.
8. **Response:** The user is provided with the predicted label, score, report path, and video link.

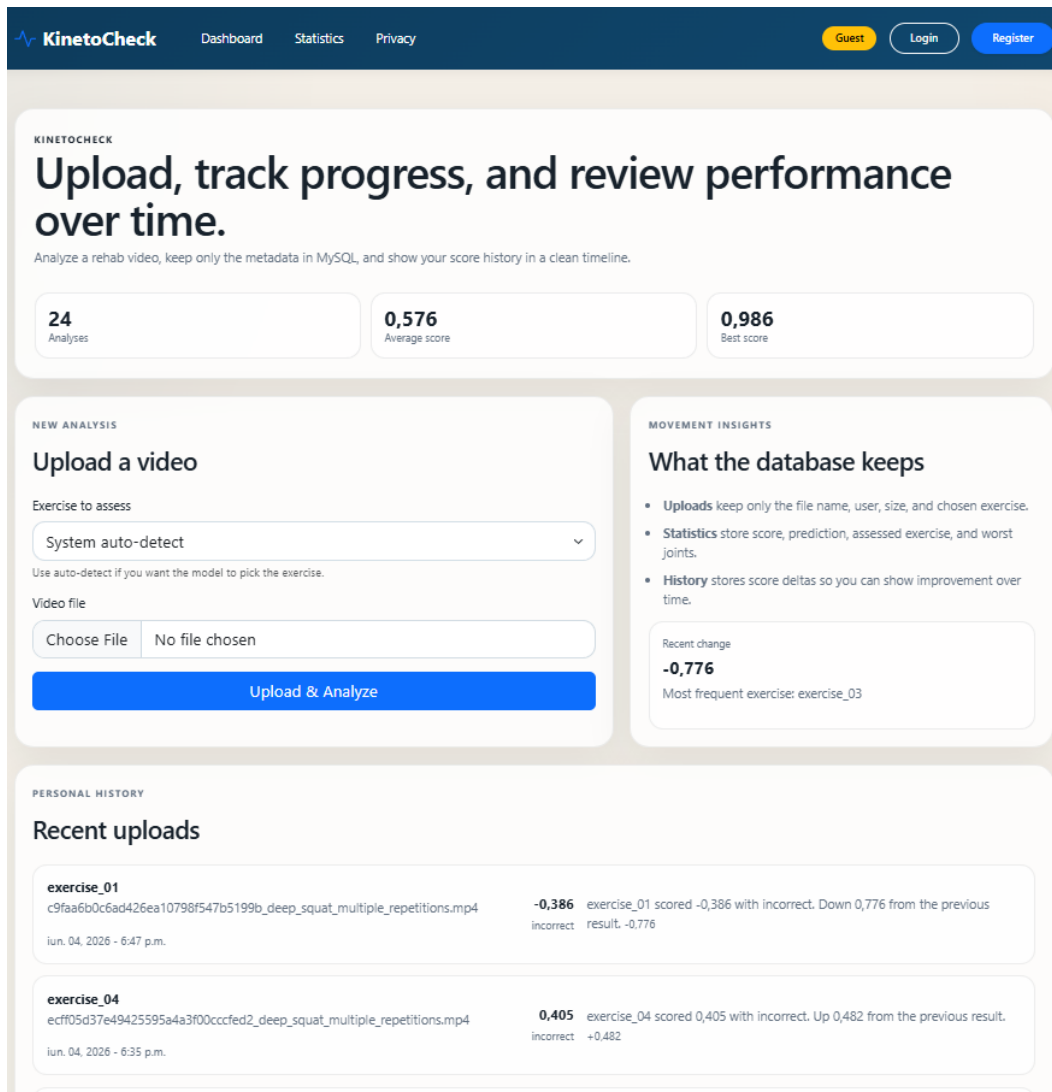


Figure 4.2: The primary user dashboard, featuring the updated navigation bar, historical statistics, and the video upload interface.

The application is designed to handle multiple concurrent requests, with asynchronous processing and caching of model checkpoints to optimize performance. Error handling is implemented at each stage to ensure that users receive informative feedback in case of issues such as invalid file formats or inference errors. The application diagram in figure 4.3 illustrates the core domain entities, data persistence layer, and primary web controllers that facilitate this workflow.

4.4.2 Data Model and Persistence (Entity Framework Core)

Although the processing core of the KinetoCheck platform relies on deep learning models, the system operates as a comprehensive web application, requiring a robust architecture to store patient data and medical rehabilitation history. Data persistence within the .NET application is managed via Entity Framework (EF) Core. This

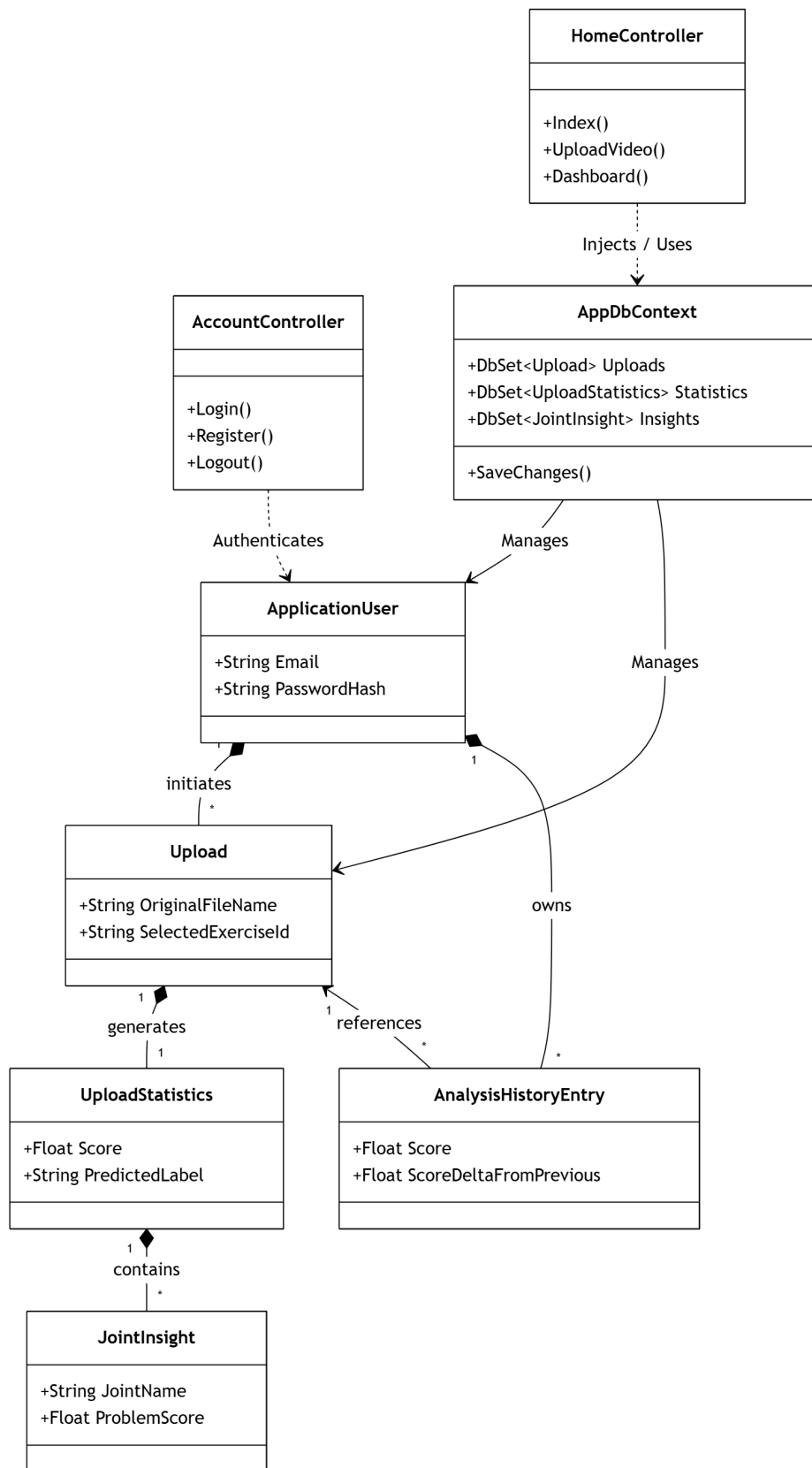


Figure 4.3: UML Class Diagram illustrating the core domain entities, data persistence layer, and primary web controllers.

modern Object-Relational Mapper (ORM) facilitates interaction with the relational database directly through C# objects (via the `AppDbContext`), eliminating the need for manual SQL query construction.

To ensure strict traceability of patient progress and the decisions made by the AI model, the database was designed following normalization principles and is structured around five primary entities (illustrated in figure 4.4):

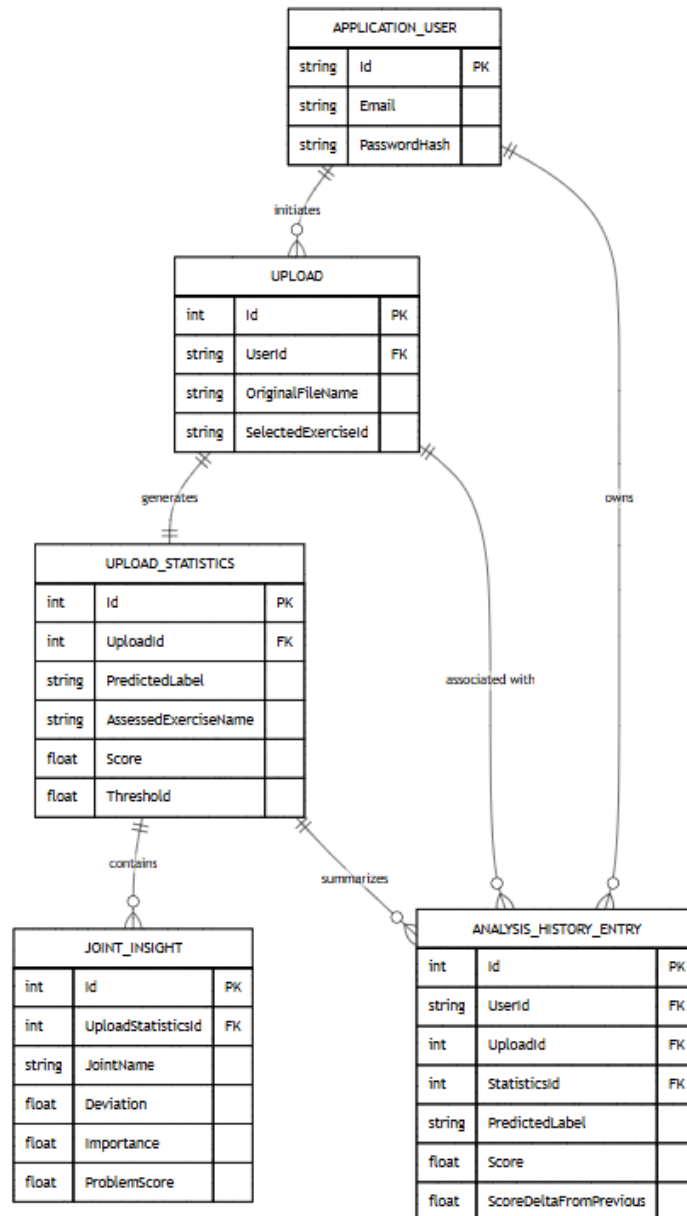


Figure 4.4: UML Class Diagram illustrating the core domain entities, data persistence layer, and primary web controllers.

- **ApplicationUser**: Extends the base `IdentityUser` class from ASP.NET Core Identity. It manages profile data and account security (e.g., encrypted password hashes, session management). By utilizing the `DeleteBehavior.SetNull`

constraint, the system prevents the accidental cascading deletion of aggregated medical data if a user account is ever deactivated.

- **Upload:** Represents the data entry point into the system. It stores the metadata of the original video file (`OriginalFileName`) and the identifier of the specific exercise selected by the user (`SelectedExerciseId`).
- **UploadStatistics:** Stores the raw inference results returned by the Python API in a strict one-to-one relationship with the `Upload` entity. This table captures the similarity score (`Score`), the model's decision (`PredictedLabel`), and the automatically calibrated decision boundaries (`Threshold`, `Margin`). Isolating this data ensures rapid query execution without overloading the primary table.
- **JointInsight:** This entity is paramount for the system's Explainable AI (XAI) capabilities. It maintains a one-to-many relationship with `UploadStatistics` and granularly stores the biomechanical errors detected at each individual joint (e.g., `Deviation`, `Importance`, `ProblemScore`). The frontend consumes this data to visually highlight problem areas (Joints of Attention) on the patient's skeleton.
- **AnalysisHistoryEntry:** Acts as a longitudinal (time-series) evolution log. Beyond storing a textual `Summary`, it captures the `ScoreDeltaFromPrevious` metric, which automatically calculates the performance difference compared to the user's previous execution of the same exercise. This feature is essential in a clinical context, allowing for the objective quantification of recovery progress over time.

The model utilizes strict Fluent API constraints—such as `MaxLength` and `Precision(6, 3)`—to optimize the database's memory footprint while ensuring the strict decimal precision required by the machine learning outputs.

4.4.3 Authentication and MVC Architecture

To guarantee the confidentiality of medical data, the platform integrates a comprehensive authentication and authorization module. The entire user interaction flow is built upon the Model-View-Controller (MVC) design pattern, native to the ASP.NET Core environment.

Authentication and Access Restriction

Access to the video analysis functionalities and medical history is strictly restricted to authenticated users. The login system, managed by the `AccountController`,

employs secure Cookie Authentication to maintain active user sessions across HTTP requests. When an unauthenticated user attempts to access a protected route marked with the `[Authorize]` attribute, the security middleware automatically intercepts the request and redirects the client to the login page, thereby preventing the exposure of sensitive data.

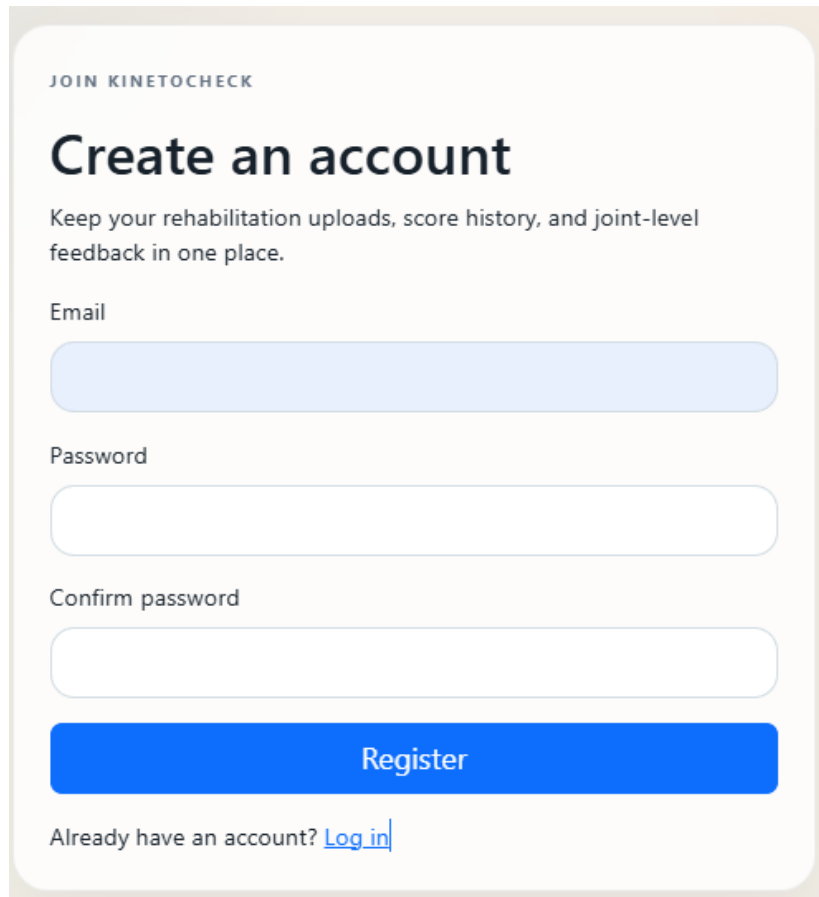
The image shows a web registration form for 'KINETO CHECK'. At the top, it says 'JOIN KINETO CHECK' in small blue letters. Below that is a large heading 'Create an account'. A subtext reads: 'Keep your rehabilitation uploads, score history, and joint-level feedback in one place.' The form contains three input fields: 'Email' (with a light blue background), 'Password', and 'Confirm password'. Below these fields is a prominent blue button labeled 'Register'. At the bottom, there is a link: 'Already have an account? [Log in](#)'.

Figure 4.5: The KinetoCheck registration interface, ensuring that medical analysis history remains secure and accessible only to registered users.

The Lifecycle of an Analysis Request (MVC)

Communication between the patient and the artificial intelligence service is elegantly orchestrated through the three components of the MVC architecture:

1. **View (Graphical Interface):** The frontend is constructed using Razor Pages (HTML5 with embedded C#). The interface includes client-side validation to ensure correct file formatting before data is transmitted to the server, preserving bandwidth and preventing unnecessary backend processing.
2. **Controller:** The application logic is cleanly separated into dedicated controllers. While the `AccountController` handles security and the

`SamplesController` manages reference data, the `HomeController` acts as the primary orchestrator for the analysis pipeline. It receives the video upload from the View, temporarily stores it, and initiates an asynchronous HTTP request to the Python analysis API (FastAPI). The asynchronous nature of this call (`async/await`) is critical, allowing the .NET server thread to remain responsive and serve other users while waiting for the neural network to finish its compute-heavy operations.

3. **Model:** Once the JSON response is returned by the AI service, the `HomeController` deserializes the payload. The data is mapped to the previously established database entities (such as `UploadStatistics` and `JointInsight`), persisted via Entity Framework Core, and the final View is rendered to display the results and the annotated ghost skeleton to the patient, thereby completing the feedback loop.

4.4.4 Error Handling and Robustness

The implementation follows the error handling specification from Chapter 3. Key error scenarios include: invalid video input, insufficient pose quality, checkpoint mismatch, and timeout conditions. For each scenario, the system provides user-friendly error messages and avoids producing ambiguous partial outputs.

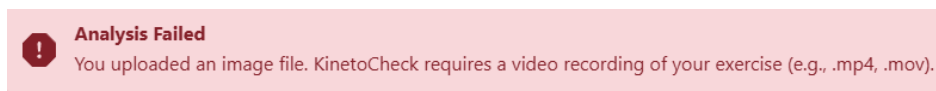


Figure 4.6: UI error message for image uploads.

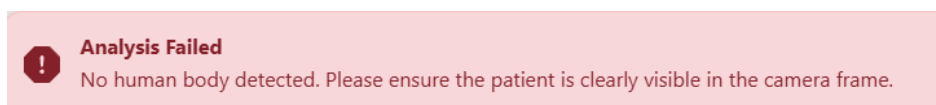


Figure 4.7: UI error message for missing human subjects.

Chapter 5

Experiments and Results

This chapter presents the empirical validation of the KinetoCheck framework through quantitative and qualitative analyses. It documents the experimental setup, reports performance metrics comparing baseline, phase-aware, and ROM-regularized variants, and provides ablation studies to isolate the contribution of each architectural component.

5.1 Experimental Setup and Protocol

The experiments reported in this chapter follow the subject-wise split protocol described in Chapter 4, where subjects are assigned to non-overlapping train, validation, and test sets. To validate strict generalization under more demanding conditions, Leave-One-Subject-Out (LOSO) evaluation is also performed as noted below.

5.2 Quantitative Analysis

The quantitative analysis compares the baseline, phase-aware, and ROM-regularized models using both classification and regression-style metrics. We report both categorical metrics (accuracy, precision, recall, F1) for binary correctness decisions and continuous metrics for quality scoring.

Table 5.2 and Table 5.3 detail the complete per-exercise evaluation metrics, including thresholds and performance bounds.

Ex. ID	Threshold	Accuracy	Precision	Recall	F1
1	0.886	0.900	0.833	1.000	0.909
2	0.060	1.000	1.000	1.000	1.000
3	0.233	0.850	1.000	0.700	0.824
4	0.801	0.900	0.833	1.000	0.909
5	-0.663	0.950	0.909	1.000	0.952
6	0.027	0.850	0.818	0.900	0.857
7	-0.387	1.000	1.000	1.000	1.000
8	-0.552	0.600	0.556	1.000	0.714
9	0.445	0.800	0.714	1.000	0.833
10	0.990	1.000	1.000	1.000	1.000

Table 5.1: Phase-aware variant with ROM (Range-of-Motion) regularization.

Ex. ID	Threshold	Accuracy	Precision	Recall	F1
1	0.758	1.000	1.000	1.000	1.000
2	0.822	1.000	1.000	1.000	1.000
3	-0.151	0.900	0.900	0.900	0.900
4	0.864	0.900	0.833	1.000	0.909
5	-0.539	0.900	0.833	1.000	0.909
6	0.697	0.950	0.909	1.000	0.952
7	0.956	1.000	1.000	1.000	1.000
8	0.601	1.000	1.000	1.000	1.000
9	0.754	0.500	0.500	1.000	0.667
10	0.880	1.000	1.000	1.000	1.000

Table 5.2: Phase-aware variant without ROM regularization.

Ex. ID	Threshold	Accuracy	Precision	Recall	F1
1	-0.415	0.850	0.769	1.000	0.870
2	0.952	1.000	1.000	1.000	1.000
3	-0.199	1.000	1.000	1.000	1.000
4	0.481	0.800	0.714	1.000	0.833
5	-0.506	1.000	1.000	1.000	1.000
6	0.663	0.950	0.909	1.000	0.952
7	0.793	0.900	1.000	0.800	0.889
8	-0.255	1.000	1.000	1.000	1.000
9	0.990	0.800	0.750	0.900	0.818
10	0.924	1.000	1.000	1.000	1.000

Table 5.3: Baseline variant.

Even though the results may not differ to much, when looking at the per-exercise metrics, the phase-aware variant is critically important when assessing videos with partial-range executions. Also, ROM regularization reduces false positives in these cases, which is clinically important for rehabilitation monitoring.

5.2.1 Dataset Characteristics

The UI-PRMD dataset contains 1,980 sequences across 10 subjects and 10 exercises, with a balanced distribution of correct and incorrect executions. Each subject performs each exercise multiple times, resulting in a rich set of movement variations. The dataset is captured using a Vicon motion capture system, providing high-fidelity 3D joint coordinates. The 39-joint Vicon captures are mapped to a 17-joint representation consistent with MediaPipe’s output, as illustrated in Figure 5.2.

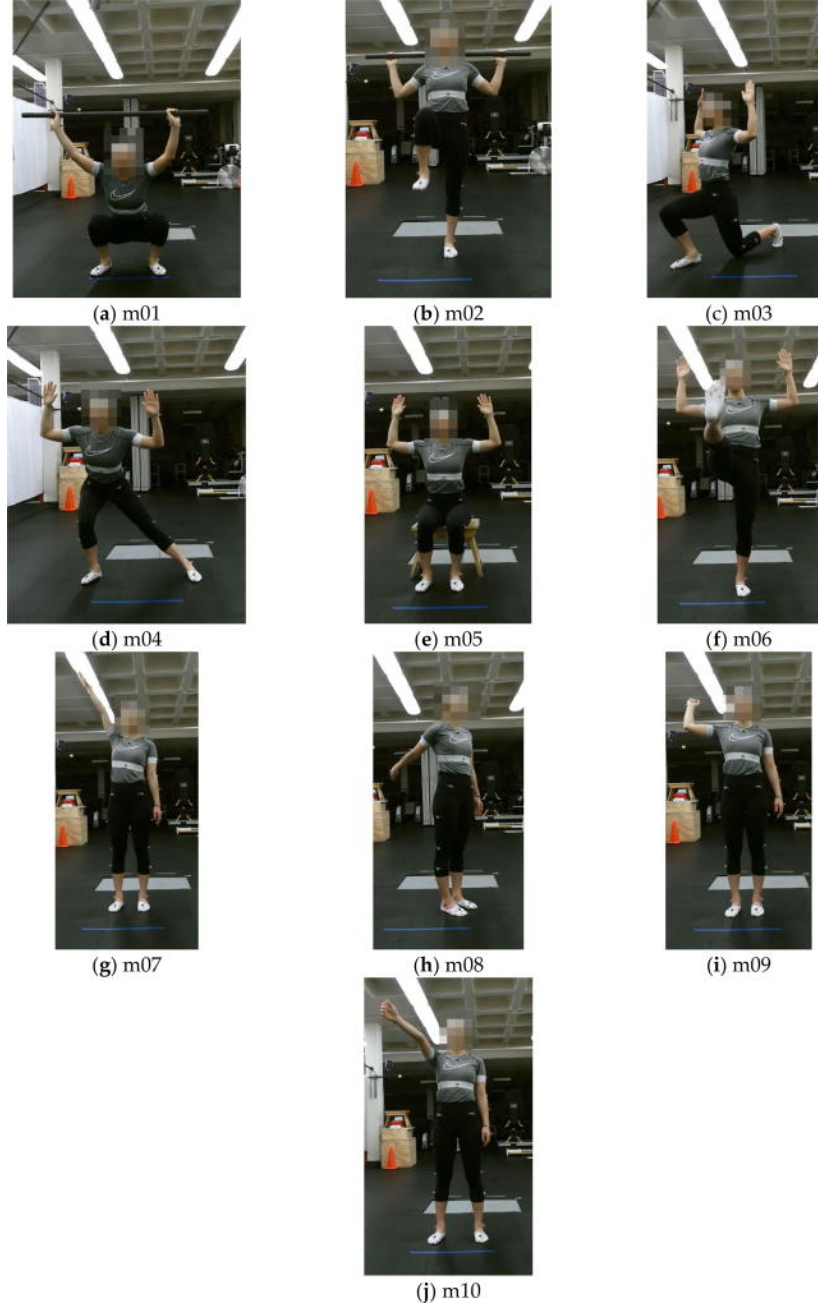


Figure 5.1: UIPRMD exercises visualization. [VJPB18]

Exercise ID	Exercise Name	Correct Count	Incorrect Count	Total Count	Subjects
1	exercise_01	110	110	220	10
2	exercise_02	110	110	220	10
3	exercise_03	99	99	198	9
4	exercise_04	110	110	220	10
5	exercise_05	110	110	220	10
6	exercise_06	110	110	220	10
7	exercise_07	110	110	220	10
8	exercise_08	110	110	220	10
9	exercise_09	110	110	220	10
10	exercise_10	110	110	220	10

Table 5.4: Dataset composition by exercise and label

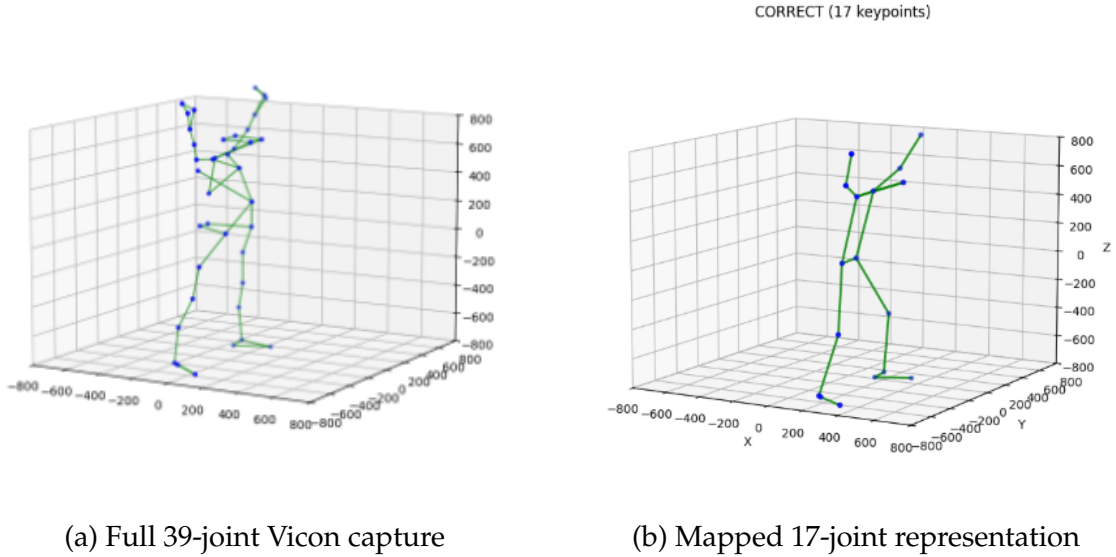


Figure 5.2: Example 3D Vicon captures showing the original marker-rich representation and the reduced 17-joint representation used in the thesis pipeline.

5.2.2 Stream Importance and Ablation Studies

We conducted ablations to measure the contribution of each component to overall performance. Results indicate that the angle-based feature strategy provides consistent gains when compared with the baseline XYZ representation. The phase-aware variant improves the reported metrics through auxiliary correction outputs, and ROM regularization reduces false positives on partial-range executions.

5.3 Qualitative Analysis

Visual inspection of the feedback overlays provides qualitative evidence that the model highlights clinically meaningful error locations. Figure 5.3 shows example visualizations for incorrect executions, comparing baseline and phase-aware variants. The baseline model doesn't expose the correct skeleton execution, while the phase-aware variant provides additional metadata that improves the interpretability of the feedback.

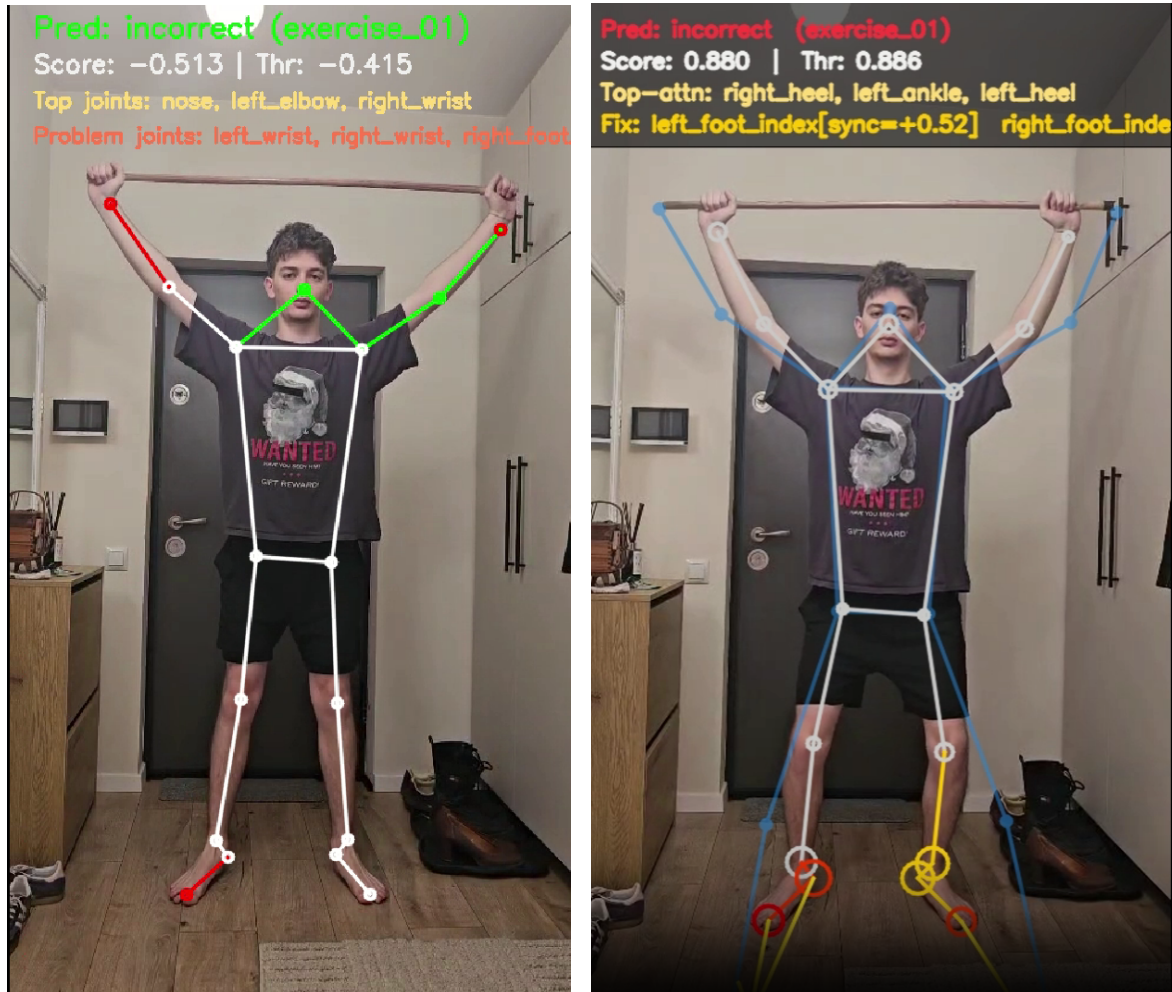


Figure 5.3: Example feedback visualizations: baseline (left) vs phase-aware (right).

5.3.1 End-to-End Clinical Feedback Walkthrough

To demonstrate the practical utility of the KinetoCheck pipeline, it is useful to examine the exact system behavior and data artifacts generated during a single rehabilitation session. Consider a scenario where a user uploads a video using the “System auto-detect” feature. In this specific case, the user attempts Exercise 01 but executes it incorrectly.

Upon processing the video, the inference engine evaluates the 12-channel kinematic sequence against all ten exercise-specific ST-GAT models concurrently. Each model computes its own similarity score and compares it against its calibrated decision threshold. The system then ranks the outputs based on the classification margin (the distance between the score and the threshold). The engine packages these insights—along with localized joint error metrics—into a structured, machine-readable JSON report. A representation of this output payload is provided in Listing 5.1.

Listing 5.1: Generated JSON report showcasing multi-model auto-detection and joint-level insights.

```

1 {
2   "video": ".../ex1-2-rep-inc.mp4",
3   "annotated_video": ".../ex1-2-rep-inc_annotated.mp4",
4   "num_frames": 236,
5   "worst_joints": [
6     {
7       "joint": "right_elbow",
8       "joint_index": 4,
9       "deviation": 0.003,
10      "importance": 0.457,
11      "problem_score": 0.966
12    },
13    {
14      "joint": "left_elbow",
15      "joint_index": 3,
16      "deviation": 0.003,
17      "importance": 0.476,
18      "problem_score": 0.981
19    },
20    { "...": "..." }
21  ],
22  "best": {
23    "exercise_id": 1,
24    "exercise_name": "exercise_02",
25    "score": 0.980,
26    "threshold": 0.060,
27    "margin": 0.920,
28    "predicted_label": "correct"
29  },
30  "all": [
31    {

```

```

32     "exercise_name": "exercise_02",
33     "score": 0.980,
34     "threshold": 0.060,
35     "margin": 0.920,
36     "predicted_label": "correct"
37 },
38 {
39     "exercise_name": "exercise_01",
40     "score": 0.880,
41     "threshold": 0.886,
42     "margin": -0.006,
43     "predicted_label": "incorrect"
44 },
45 { "...": "..." }
46 ]
47 }

```

As illustrated in Listing 5.1, the model does not merely return a binary pass/fail state. The `all` array reveals the multi-task evaluation: while the system identified the movement as a passing execution of Exercise 02 (yielding the highest positive margin of 0.920), it correctly flagged the execution as `incorrect` when evaluated strictly against the Exercise 01 template (failing the threshold by a margin of -0.006).

Furthermore, the payload isolates the anatomical source of the error within the `worst_joints` array. In this execution, the left and right elbows are explicitly flagged as the primary components degrading the similarity score. The `importance` metric is derived directly from the Spatial Graph Attention (ST-GAT) weights, while the `deviation` magnitude is provided by the auxiliary Frame Decoder.

The ASP.NET web interface parses this payload, mapping the data to the `UploadStatistics` and `JointInsight` database entities. It then renders the user dashboard, overlaying the ghost skeleton on the video and drawing explicit visual attention to the failing joints. This pipeline successfully transforms complex, multi-model neural network outputs into a localized, interpretable, and highly actionable corrective cue for the patient.

5.3.2 Longitudinal Patient Tracking and Analytics

While isolated frame-by-frame feedback is critical for immediate postural correction, the ultimate goal of a physical rehabilitation platform is to quantify recovery over time. To serve both the patient and the overseeing physical therapist, KinetoCheck aggregates the `AnalysisHistoryEntry` database records into a comprehensive clinical statistics dashboard.

As shown in Figure 5.4, the system processes historical uploads to visualize the patient’s recovery trajectory. By aggregating metrics such as the total volume of exercises performed, the overall success rate, and a time-series plot of recent similarity scores, the platform provides objective proof of motor function improvement. Furthermore, by isolating the most frequently failing anatomical joints across all sessions, the system allows therapists to identify chronic biomechanical weaknesses and adjust the patient’s training regimen accordingly.



Figure 5.4: The KinetoCheck statistics dashboard, providing longitudinal analytics, success rates, and chronic joint error aggregations for long-term rehabilitation tracking.

5.4 Generalization Challenges and Limitations

Although results are promising, several limitations remain important to acknowledge. The primary limitation is dataset size. UI-PRMD contains 1,980 sequences across 10 subjects and 10 exercises, which is modest compared to large-scale action recognition benchmarks.

As discussed in Chapter 2, attention-based explanations should be interpreted cautiously. High attention does not automatically imply causal biomechanical error. The feedback overlays provide a useful heuristic but are not a substitute for expert clinical judgment.

5.4.1 Error Analysis and Known Failure Modes

While the quantitative metrics demonstrate high classification performance, a rigorous error analysis reveals specific edge cases where the pipeline struggles. Understanding these failure modes is critical for establishing safe clinical deployment boundaries. The primary failure modes observed during qualitative testing include:

- **Severe Self-Occlusion:** In exercises requiring deep torso rotation or limb crossing, the MediaPipe Pose Landmarker occasionally loses track of distal joints (e.g., wrists and ankles). Although the temporal imputation layer forward-fills short dropouts (< 2 frames), extended occlusions result in rigid, frozen joint coordinates. The ST-GAT model, interpreting this lack of movement as a biomechanical error, often triggers a false positive for incorrect execution.
- **Depth Ambiguity in 2D Projections:** Because the system extracts 3D coordinates from a 2D RGB feed, movements occurring perfectly parallel to the camera’s optical axis (e.g., lunging directly toward the lens) suffer from depth compression. The Z-coordinate variance drops significantly, which can occasionally cause the model to underestimate the Range of Motion (ROM) and penalize a physically correct execution.
- **Clothing-Induced Landmark Jitter:** Loose or highly textured clothing can confuse the underlying pose extraction heuristics, causing the inferred skeletal landmarks to jitter rapidly between frames. While the preprocessing layer applies Gaussian temporal smoothing to mitigate this, extreme high-frequency jitter is sometimes encoded by the temporal pyramid’s short-term convolutional kernels (kernel size=3) as muscle tremors or instability, negatively skewing the similarity score.

These failure modes underscore the necessity of the “Joints of Attention” feedback overlays. By explicitly visualizing the ghost skeleton and highlighting the problem-

atic joints, the system allows the user to immediately recognize if a low score was caused by a genuine form error or a temporary pose-extraction failure.

Chapter 6

Conclusions and Future Directions

This chapter summarizes the contributions of the thesis and reflects on their significance within the broader context of rehabilitation technology and machine learning. It recapitulates how the work addresses the research gaps identified in Chapter 2, consolidates the key findings from the experimental validation in Chapter 5, and outlines promising directions for future research and practical deployment.

6.1 Thesis Contributions Summary

This thesis has developed *KinetoCheck*, a comprehensive framework for interpretable, subject-invariant rehabilitation exercise quality assessment. The key contributions span four complementary areas:

Representation and Model Design: Proposed and implemented systematic geometric normalization and preprocessing strategies that enable reliable transfer from laboratory motion-capture training (UI-PRMD) to real-world RGB-based deployment via MediaPipe. Organized the implementation around feature strategies, allowing the training pipeline to switch between baseline XYZ features and angle-augmented features.

Subject-Invariant Training and Evaluation: Applied rigorous subject-level split protocols (subject-wise and Leave-One-Subject-Out) to ensure that models generalize to unseen subjects rather than overfitting to subject-specific body characteristics. Centralized the training loop in a reusable trainer so that baseline, factory-based, and phase-aware runs share the same evaluation and checkpointing logic.

Interpretability and Clinical Actionability: Extracted and operationalized attention-based explanations as joint-level feedback overlays (Joints of Attention), providing localized corrective hints that guide users and therapists toward movement improvements. Designed a standalone analytics service that aggregates individual inference results into user-level progress metrics, enabling longitudinal

tracking.

End-to-End System Integration: Implemented a modular, maintainable system that separates training, inference, and reporting components. Engineered checkpoint management and factory-pattern loading that gracefully switches between baseline, phase-aware, and ROM-regularized model variants. Implemented comprehensive error detection and user-transparent fallback behaviors for common failure modes.

6.2 Key Empirical Findings

The experimental validation in Chapter 5 yields several important findings across three dimensions:

Performance Improvements: The comparison in Chapter 5 shows distinct performance differences between the baseline, and phase-aware variants. LOSO evaluation, when used, produces more conservative estimates than subject-wise splits, confirming the necessity of rigorous split protocols.

Interpretability Validation: Qualitative analysis of feedback overlays demonstrates that the model highlights clinically meaningful error locations, lending credibility to the interpretability claims. The phase-aware variant exposes additional correction metadata, which improves the readability of the feedback presented to the user.

Generalization and Robustness: Subject-invariant training via geometric normalization and augmentation effectively mitigates identity leakage. ROM regularization provides robustness against partial-range executions, reducing false positives in clinical scenarios where full range-of-motion is medically necessary.

6.3 Limitations and Honest Assessment

While the contributions are substantial, several limitations warrant explicit discussion. The UI-PRMD dataset is modest in scale; future work should validate generalization across broader demographic ranges and exercise repertoires. Furthermore, the thesis has focused primarily on offline evaluation from pre-recorded videos. Real-time performance, user interface design, and clinician feedback integration remain important practical considerations.

Finally, as discussed in Chapter 2, attention mechanisms are heuristics, not causal explanations. The feedback overlays provide useful decision support but should not be interpreted as definitive biomechanical diagnoses.

6.4 Future Research Directions

Several promising directions emerge from this work:

- **Multi-Modal Integration:** Extending the framework to fuse RGB and depth data to improve robustness in challenging lighting or occlusion scenarios.
- **Edge Deployment:** Quantizing the model for mobile devices to enable decentralized home rehabilitation monitoring.

6.5 Concluding Remarks

This thesis has presented a comprehensive framework for interpretable, subject-invariant rehabilitation exercise assessment. By combining rigorous machine learning practices with practical system engineering, KinetoCheck demonstrates that high-performance rehabilitation AI is achievable without sacrificing clinical transparency.

As rehabilitation technology continues to evolve, this thesis contributes both concrete tools and methodological insights that will hopefully inform future work in clinical AI. The code, data, and detailed documentation provided are offered in the spirit of open science, with the hope that they accelerate progress in rehabilitation technology.

Bibliography

- [AGM⁺18] Julius Adebayo, Justin Gilmer, Michael Muelly, Ian Goodfellow, Moritz Hardt, and Been Kim. Sanity checks for saliency maps. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.
- [BGL⁺93] Jane Bromley, Isabelle Guyon, Yann LeCun, Eduard Sackinger, and Roopak Shah. Signature verification using a "Siamese time delay neural network. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 6, 1993.
- [BGR⁺20] Valentin Bazarevsky, Ivan Grishchenko, Karthik Raveendran, Tyler Zhu, Fan Zhang, and Matthias Grundmann. Blazepose: On-device real-time body pose tracking, 2020.
- [CSWS17] Zhe Cao, Tomas Simon, Shih-En Wei, and Yaser Sheikh. Realtime multi-person 2d pose estimation using part affinity fields. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7291–7299, 2017.
- [CXJS22] Shuo Chen, Ke Xu, Xinghao Jiang, and Tanfeng Sun. Pyramid spatial-temporal graph transformer for skeleton-based action recognition. *Applied Sciences*, 12(18):9229, 2022.
- [GYDD18] Pallabi Ghosh, Yi Yao, Larry S. Davis, and Ajay Divakaran. Stacked spatio-temporal graph convolutional networks for action segmentation, 2018.
- [HCL06] Raia Hadsell, Sumit Chopra, and Yann LeCun. Dimensionality reduction by learning an invariant mapping. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1735–1742, 2006.
- [JW19] Sarthak Jain and Byron C. Wallace. Attention is not explanation. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 3543–3556, 2019.

- [KZS15] Gregory Koch, Richard Zemel, and Ruslan Salakhutdinov. Siamese neural networks for one-shot image recognition. In *ICML Deep Learning Workshop*, 2015.
- [LTN⁺19] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Yang, Zhiyuan Zhang, Chuo-Ling Chang, Ming Yong, Juhyun Lee, Wan-Teh Chang, Wei Hua, Matthias Georg, Maniam Kammel, David Shearer, Karen Wang, Mingze Wang, Oscar Ramirez, Jeff Weiss, Vinit Jain, et al. Mediapipe: A framework for building perception pipelines, 2019.
- [Rud19] Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215, 2019.
- [VJPB18] Aleksandar Vakanski, Hyung-pil Jun, David Paul, and Russell Baker. A data set of human body movements for physical rehabilitation exercises. *Data*, 3(1):2, 2018.
- [ZGAA25] Moamen Zaher, Amr S. Ghoneim, Laila Abdelhamid, and Ayman Atia. Rehabilitation monitoring and assessment: a comparative analysis of feature engineering and machine learning algorithms on the ui-prmd and kimore benchmark datasets. *Sports Engineering*, 28(1):382–402, 2025. Published online 04 Feb 2025.